ТЕХНІЧНИЙ ОПИС ПРОЄКТУ

Інтернет-магазин косметики

Дипломна робота

Гуцол А. А.

2026

1. Загальна архітектура проєкту

Проєкт побудований за класичною клієнт-серверною архітектурою (SPA + REST API). Фронтенд реалізований на React та працює як Single Page Application — тобто сторінки не перезавантажуються, а весь перехід між розділами відбувається на стороні клієнта. Бекенд — Node.js з фреймворком Express — виконує бізнес-логіку та надає дані через REST API. Зберігання всіх даних здійснюється в реляційній базі даних MySQL.

Рівень	Технологія	Призначення
Frontend	React 18, React Router v6	Інтерфейс користувача, SPA-навігація
Backend	Node.js, Express	REST API, бізнес-логіка
База даних	MySQL	Зберігання всіх даних додатку
Зв'язок	HTTP/JSON (Fetch API)	Обмін даними між клієнтом і сервером

2. Структура файлів та призначення кожного

2.1. Кореневі файли

server.js

Серце всього бекенду. Це найбільший файл проєкту (~830 рядків коду, близько 55 API-маршрутів). Тут описана вся серверна логіка: реєстрація та авторизація користувачів, управління товарами, замовленнями, промокодами, підтримкою, сповіщеннями. Сервер запускається на порті 5000 і приймає запити від React-додатку.

- Підключає всі npm-пакети та middleware
- Визначає JWT-секрет та email-транспортер (nodemailer)
- Містить усі функції middleware для захисту маршрутів
- Реалізує всі REST API-ендпоінти

db.js

Модуль підключення до MySQL. Використовує пул з'єднань (mysql2/promise) — це означає, що замість відкриття нового з'єднання на кожен запит, сервер використовує вже відкриті з'єднання повторно. Пул автоматично управляє кількістю з'єднань, що суттєво покращує продуктивність під навантаженням.

init_db.sql

SQL-скрипт для первинного розгортання бази даних. Містить команди CREATE TABLE для всіх 15 таблиць, а також INSERT із тестовими даними (кілька демо-товарів і промокод WELCOME10). Виконується один раз при налаштуванні серверу.

.env

Файл конфігурації середовища. Зберігає чутливі параметри: JWT_SECRET (ключ підпису токенів), дані підключення до БД, SMTP-налаштування для відправки листів, URL фронтенду. Цей файл не потрапляє до системи контролю версій з міркувань безпеки.

2.2. Фронтенд (src/)

src/App.js

Головний компонент React-додатку. Виконує декілька критично важливих функцій одночасно. По-перше, тут зберігається весь глобальний стан: кошик, список вибраного (wishlist), список порівняння, дані авторизованого користувача, сповіщення. По-друге, тут налаштований весь клієнтський роутинг через React Router. По-третє, тут розміщені компоненти навбару (включно з пошуком та дзвіночком сповіщень) і футера.

- Кошик синхронізується з localStorage через useEffect
- Пошук у навбарі має debounce 260мс для оптимізації запитів
- Дзвіночок сповіщень опитує сервер кожні 30 секунд
- Містить компонент ProtectedRoute для захисту закритих сторінок

src/App.css

Єдиний великий файл стилів (~1600 рядків). Містить CSS-змінні для теми оформлення (кольори, радіуси, відступи), стилі для всіх компонентів, медіа-запити для адаптивності на мобільних пристроях та анімації.

src/index.js

Точка входу React-додатку. Монтує кореневий компонент App у DOM-елемент з id="root".

2.3. Компоненти (src/components/)

Catalog.jsx

Головна сторінка каталогу товарів. Відображає список товарів у вигляді сітки. Підтримує фільтрацію за категорією, текстовий пошук, сортування (за ціною, рейтингом, новизною), фільтр за діапазоном цін та фільтр за атрибутами продуктів. Усі фільтри передаються як GET-параметри до API.

ProductCard.jsx

Компонент картки окремого товару в каталозі. Відображає головне фото, назву, поточну ціну (або перекреслену ціну зі знижкою), середній рейтинг, кнопки додавання в кошик, вибране та список порівняння.

ProductPage.jsx

Повна сторінка товару. Включає галерею фотографій з можливістю перемикання, повний опис, таблицю характеристик (атрибутів), блок відгуків покупців із зірковим

рейтингом, блок питань та відповідей (Q&A), а також секцію нещодавно переглянутих товарів.

Cart.jsx

Сторінка кошика. Показує список доданих позицій, дозволяє змінити кількість кожного товару або видалити його. Відображає загальну суму та кнопку переходу до оформлення.

Checkout.jsx

Багатокрокова форма оформлення замовлення. Реалізований покроковий процес: вибір способу доставки (Нова Пошта, Укрпошта, Кур'єр — кожен з відповідними полями), введення контактних даних отримувача, вибір способу оплати, введення промокоду та підсумок замовлення. Підтримує автодоповнення міст.

Profile.jsx

Особистий кабінет. Поєднує функціональність авторизації (форми входу і реєстрації) та управління профілем (редагування ПІБ, телефону, дати народження, збереженої адреси). Також відображає список власних замовлень і список вибраного.

OrderPage.jsx

Сторінка детального перегляду конкретного замовлення. Відображає список товарів, суму, статус, трекінг-номер та дату оформлення.

AdminPanel.jsx

Панель управління товарним каталогом для адміністратора та менеджера. Дозволяє додавати нові товари, редагувати існуючі та видаляти. Підтримує завантаження до 8 фотографій на товар. Містить модальне вікно для управління словником характеристик (атрибутів).

AdminOrders.jsx

Панель управління замовленнями. Відображає всі замовлення з можливістю фільтрації за статусом. Дозволяє персоналу змінювати статус замовлення та вносити трекінг-номер.

AdminPromo.jsx

Панель управління промокодами (тільки адмін). Відображає список промокодів, дозволяє створювати нові різних типів, активувати/деактивувати та видаляти.

AdminUsers.jsx

Панель управління користувачами (тільки адмін). Відображає список зареєстрованих користувачів, дозволяє змінювати роль (user/manager/admin).

SalesPage.jsx

Сторінка акцій. Показує лише товари, у яких є активна знижка (discount_price != null та discount_expiry в майбутньому).

ComparePage.jsx

Сторінка порівняння товарів. Відображає обрані для порівняння товари у вигляді горизонтальної таблиці, де кожен стовпець — це товар, а рядки — характеристики.

SupportPage.jsx

Сторінка списку тікетів підтримки. Для звичайного користувача відображає тільки його тікети. Для персоналу (admin/manager) — всі тікети з можливістю фільтрації за статусом. Дозволяє створювати нові тікети.

SupportTicket.jsx

Сторінка конкретного тікету. Реалізує чат-інтерфейс між користувачем і персоналом підтримки. Персонал може змінювати статус тікету (open/answered/closed).

Toast.jsx

Компонент спливаючих повідомлень. Відображається на 3.5 секунди у нижньому куті екрану. Підтримує два типи: success (зелений) та error (червоний).

3. Технології та методи розробки

3.1. Backend-технології

Пакет/технологія	Призначення та метод використання
Node.js + Express	Веб-сервер. Express реалізує REST API через app.get/post/put/delete/patch маршрути
mysql2/promise	Робота з MySQL через промісифікований пул з'єднань. Всі запити через await db.query()
bcryptjs	Хешування паролів. При реєстрації: bcrypt.hash(password, 10). При вході: bcrypt.compare()
jsonwebtoken	Генерація та верифікація JWT-токенів. jwt.sign() при вході, jwt.verify() в middleware
multer	Обробка завантаження файлів (зображення товарів). diskStorage з унікальними іменами файлів
nodemailer	Відправка email через SMTP (Gmail). Для скидання паролю та сповіщень персоналу
cors	Cross-Origin Resource Sharing — дозволяє фронтенду (порт 3000) звертатись до API (порт 5000)
ES Modules	Сучасний синтаксис import/export замість CommonJS require()

3.2. Frontend-технології

Технологія	Призначення та метод використання
React 18	Компонентний підхід до побудови UI. Хуки: useState (стан), useEffect (побічні ефекти), useCallback (мемоізація функцій), useRef (посилання на DOM)
React Router v6	Клієнтський роутинг. Routes/Route для маршрутів, Navigate для перенаправлень, useNavigate/useLocation/useParams для навігації
Fetch API	Нативні браузерні HTTP-запити до REST API. Використовується без зовнішніх бібліотек (без axios)
CSS Variables	Темізація через змінні: --gold, --bg, --card, --border, --t1, --t2. Дозволяє легко змінювати кольорову схему
localStorage	Збереження стану між сесіями: кошик (cart), дані користувача (user), session_id, нещодавно переглянуті (recent)

3.3. Методи розробки бекенду

Транзакції MySQL

При оформленні замовлення використовується повноцінна SQL-транзакція (BEGIN TRANSACTION / COMMIT / ROLLBACK). Це гарантує атомарність операції: якщо під час оформлення виникає будь-яка помилка (наприклад, товар щойно закінчився), жоден рядок не буде збережений у БД, і залишки товарів не зміняться.

SELECT FOR UPDATE

При перевірці залишку товару в транзакції замовлення виконується SELECT ... FOR UPDATE — це блокує відповідний рядок таблиці products на час транзакції. Завдяки цьому два користувачі не зможуть одночасно купити один і той самий останній товар (race condition виключений).

Динамічне формування SQL-запитів

Запит для каталогу товарів будується програмно залежно від переданих параметрів фільтрації. Кожен атрибут фільтра додає окремий JOIN до таблиці product_attrs, умови WHERE збираються в рядок із масивом параметрів. Це дозволяє гнучко комбінувати фільтри без дублювання SQL-коду.

Middleware-ланцюжки

Express middleware використовуються як захисні прошарки перед обробниками маршрутів. Кожен маршрут може мати один або кілька middleware: authMiddleware перевіряє токен, потім adminMiddleware або staffMiddleware перевіряє роль. Це дозволяє чітко розмежувати доступ.

4. Система авторизації та захист

4.1. JWT-токени

При успішному вході або реєстрації сервер генерує JSON Web Token (JWT). Токен підписується секретним ключем (JWT_SECRET з .env файлу) і містить дані користувача: id та role. Термін дії токену — 24 години.

Фронтенд зберігає токен у localStorage та передає його в HTTP-заголовку кожного захищеного запиту у форматі: Authorization: Bearer <token>.

Для скидання паролю генерується окремий токен з додатковим полем purpose: 'reset' — він діє лише 1 годину і може бути використаний виключно для зміни паролю (перевіряється поле purpose при верифікації).

4.2. Middleware для захисту маршрутів

Middleware	Опис та де застосовується
authMiddleware	Перевіряє наявність і валідність JWT-токену. Якщо токен відсутній — 403, прострочений/невалідний — 401. Застосовується до всіх захищених маршрутів
adminMiddleware	authMiddleware + перевірка role === 'admin'. Застосовується до управління промокодами, списком користувачів, зміни ролей
staffMiddleware	authMiddleware + перевірка role IN ('admin','manager'). Для статистики, тікетів підтримки, атрибутів
canManageProducts	Аналогічний staffMiddleware. Для CRUD-операцій над товарами
canManageOrders	Аналогічний staffMiddleware. Для управління замовленнями
optionalAuth	Токен необов'язковий. Якщо є — розбирає і передає user далі. Для каталогу, перегляду товару, порівняння

4.3. Хешування паролів

Паролі ніколи не зберігаються у відкритому вигляді. При реєстрації пароль хешується функцією bcrypt.hash(password, 10) — число 10 означає кількість "раундів соління", що визначає складність обчислення хешу. При авторизації bcrypt.compare() порівнює введений пароль з хешем у БД без потреби дешифрування.

4.4. Захист від витоку даних

При запиті на скидання паролю сервер завжди повертає однакову відповідь: "Якщо такий email існує — лист надіслано" — незалежно від того, чи зареєстрований цей email. Це унеможливлює перебір email-адрес (email enumeration attack).

4.5. Захист завантажуваних файлів

Multer перевіряє MIME-тип завантажуваного файлу через fileFilter — дозволяються лише jpeg/jpg/png/webp. Максимальний розмір файлу обмежений 8 МБ

(limits.fileSize). Ім'я файлу замінюється на унікальне (timestamp + random) — це захищає від перезапису існуючих файлів.

4.6. Ролева система доступу (RBAC)

Роль	Доступні функції
user	Перегляд каталогу, оформлення замовлень, вибране, порівняння, відгуки, питання, підтримка, особистий кабінет
manager	Все що user + управління товарами (CRUD), управління замовленнями (статуси, трекінг), відповіді на питання та відгуки, ведення чату підтримки
admin	Все що manager + управління промокодами, управління користувачами та їх ролями, перегляд статистики

5. Структура бази даних

База даних MySQL cosmetics_shop містить 15 таблиць. Всі таблиці використовують кодування utf8mb4 для повної підтримки Unicode. Між таблицями визначені зовнішні ключі (FOREIGN KEY) для забезпечення цілісності даних.

Таблиця	Призначення та ключові поля
users	Акаунти користувачів: full_name, email (унікальний), password_hash, phone, role (ENUM: user/manager/admin), saved_address, birthday
products	Товари: title, description, price, discount_price, discount_expiry, stock_quantity, image_url, category
product_images	Додаткові фото товару. Зовнішній ключ product_id з CASCADE DELETE
product_attrs	Характеристики товарів: attr_name, attr_value. Дозволяє гнучко додавати будь-які атрибути без зміни схеми
attr_options	Словник допустимих значень характеристик. Унікальна пара (attr_name, attr_value). Використовується у формі додавання товару
reviews	Відгуки: product_id, user_id, user_name, rating (1-5, CHECK constraint), body
product_questions	Q&A до товарів: question, answer, answered_by, answered_at. answer може бути NULL (ще не відповіли)
orders	Замовлення: user_id (nullable для гостей), full_name, phone, address, total_price, delivery_method, payment_method, status (ENUM), tracking_number
order_items	Позиції замовлення: order_id, product_id, quantity, price_at_purchase. Ціна фіксується на момент замовлення
support_tickets	Тікети підтримки: user_id, user_name, user_email, subject, status (ENUM: open/answered/closed)
support_messages	Повідомлення в тікеті: ticket_id, sender_id, sender_name, sender_role, body
wishlist	Список вибраного: user_id + product_id (унікальна пара). Потребує авторизації
compare_list	Список порівняння: session_id + product_id (унікальна пара). Не потребує авторизації — ідентифікація за session_id
promo_codes	Промокоди: code, discount_percent, promo_type, max_uses, uses_count, expires_at, min_order_amount, is_active
promo_usage	Фіксує, хто і коли використав промокод. Для типів single_use та first_order
notifications	Сповіщення: user_id, type, message, order_id (nullable), is_read. Для персоналу та покупців

6. Основні механіки та бізнес-логіка

6.1. Механіка кошика

Кошик зберігається виключно на стороні клієнта в localStorage браузера. Це означає, що кошик зберігається навіть без авторизації та між сесіями. Стан кошика в React синхронізується з localStorage через useEffect: кожного разу при зміні масиву cart дані автоматично записуються.

При додаванні товару перевіряється поточна кількість у кошику: якщо вже додана кількість плюс нова перевищує stock_quantity — операція блокується з повідомленням помилки.

6.2. Промокоди — типи та механіка

Система підтримує 4 різних типи промокодів зі своєю логікою перевірки:

- standard — базовий відсотковий промокод, доступний всім
- first_order — тільки для першого замовлення. Сервер перевіряє через SELECT чи є вже замовлення з цим user_id
- single_use — кожен авторизований користувач може використати лише раз. Факт використання записується в таблицю promo_usage
- birthday — діє тільки в місяць народження користувача. Порівнюється getMonth() поточної дати з getMonth() users.birthday
- min_amount — активується тільки якщо сума замовлення перевищує min_order_amount

Важливо: перевірка промокоду відбувається двічі. Спочатку при натисканні кнопки "Застосувати" (превентивна перевірка для UX), а потім повторно всередині SQL-транзакції при фінальному оформленні замовлення — це захищає від маніпуляцій на клієнті.

6.3. Механіка оформлення замовлення

Оформлення замовлення повністю обгорнуте в SQL-транзакцію:

- 1. Для кожного товару: SELECT ... FOR UPDATE (блокування рядка)
- 2. Перевірка stock_quantity — якщо недостатньо, одразу ROLLBACK
- 3. Визначення актуальної ціни (звичайна або зі знижкою якщо discount_expiry в майбутньому)
- 4. Перевірка і застосування промокоду (якщо є)
- 5. INSERT в таблицю orders
- 6. INSERT в order_items для кожної позиції із зафіксованою ціною
- 7. UPDATE products — зменшення stock_quantity
- 8. Фіксація використання промокоду (якщо single_use/first_order)
- 9. COMMIT — тільки якщо всі кроки успішні

При будь-якій помилці на будь-якому кроці виконується ROLLBACK — жоден рядок не зберігається. Замовлення можуть оформляти навіть незареєстровані користувачі (user_id = NULL).

6.4. Система сповіщень

Функція notifyStaff() виконує два кроки: спочатку INSERT в таблицю notifications для кожного адміна та менеджера, потім (якщо налаштований SMTP) відправляє email на їх адреси. Сповіщення з'являються в дзвіночку навбару, який опитує

/api/notifications кожні 30 секунд (setInterval). Непрочитані сповіщення виділяються, кількість відображається на бейджі.

6.5. Пошук та фільтрація

Пошук у навбарі реалізований з debounce 260мс: запит до API відправляється не на кожну набрану букву, а через 260 мс після зупинки введення. Це зменшує навантаження на сервер. Результати відображаються у випадаючому списку прямо під рядком пошуку.

Фільтрація в каталозі підтримує: категорію, текстовий пошук (LIKE по title і description), діапазон цін (min_price/max_price), фільтр по знижці та фільтр по атрибутах. SQL-запит будується динамічно: для кожного атрибута додається окремий JOIN до product_attrs.

6.6. Порівняння товарів

Список порівняння зберігається в БД у таблиці compare_list за session_id — рядком, який генерується при першому відкритті сайту і зберігається в localStorage. Авторизація не потрібна. Максимальна кількість товарів для порівняння обмежена 4 позиціями на сервері.

6.7. Завантаження зображень

Multer зберігає завантажені файли в папку uploads/ на сервері. Ім'я файлу формується як timestamp-random.ext (наприклад, 1712345678901-482947.jpg) — це гарантує унікальність і запобігає перезапису. Максимальний розмір — 8 МБ. Дозволені формати: jpeg, jpg, png, webp. Статика роздається через express.static на шляху /uploads.

6.8. Email-функціональність

Nodemailer налаштований для роботи з Gmail SMTP (порт 587, STARTTLS). Листи надсилаються у двох випадках: при запиті скидання паролю (на адресу користувача з посиланням, що містить JWT-токен, дійсний 1 годину) та при нових замовленнях (персоналу). Якщо SMTP не налаштований у .env, система продовжує коректно працювати — лише без відправки листів.

7. REST API — перелік маршрутів

7.1. Авторизація (/api/auth)

Метод	Маршрут	Опис
POST	/api/auth/register	Реєстрація. Хешує пароль, повертає JWT
POST	/api/auth/login	Вхід. Перевіряє bcrypt, повертає JWT
GET	/api/auth/me	Дані поточного користувача (authMiddleware)
PUT	/api/auth/me	Оновлення профілю (authMiddleware)
POST	/api/auth/forgot-password	Надсилає лист зі скиданням паролю
POST	/api/auth/reset-password	Встановлює новий пароль за reset-токеном

7.2. Товари (/api/products)

Метод	Маршрут	Опис
GET	/api/products	Каталог з фільтрами (optionalAuth)
GET	/api/products/:id	Один товар (optionalAuth)
GET	/api/categories	Список категорій
POST	/api/products	Додати товар (canManageProducts)
PUT	/api/products/:id	Оновити товар (canManageProducts)
DELETE	/api/products/:id	Видалити товар (canManageProducts)
GET/POST	/api/products/:id/reviews	Відгуки до товару
GET/POST	/api/products/:id/questions	Q&A до товару

7.3. Замовлення, промокоди, адмін

Метод	Маршрут	Опис
POST	/api/promo/check	Перевірка промокоду (optionalAuth)
POST	/api/orders	Оформити замовлення (optionalAuth, транзакція)
GET	/api/orders/my	Свої замовлення (authMiddleware)
GET	/api/orders/:id	Деталі замовлення (authMiddleware)
GET/POST	/api/wishlist	Вибране (authMiddleware)

GET	/api/admin/stats	Статистика (staffMiddleware)
GET	/api/admin/orders	Всі замовлення (canManageOrders)
PATCH	/api/admin/orders/:id	Змінити статус замовлення
GET	/api/admin/users	Список користувачів (adminMiddleware)
PATCH	/api/admin/users/:id/role	Змінити роль (adminMiddleware)
GET/POST	/api/admin/promo	Управління промокодами (admin)
GET/POST/PATCH	/api/notifications	Сповіщення (authMiddleware)

8. Клієнтський роутинг

React Router v6 керує навігацією без перезавантаження сторінки. Компонент ProtectedRoute перевіряє наявність авторизації та роль користувача — якщо доступ заборонений, відбувається перенаправлення на головну сторінку.

Маршрут	Доступ	Компонент
/	Всі	Catalog — каталог товарів
/product/:id	Всі	ProductPage — сторінка товару
/cart	Всі	Cart — кошик
/checkout	Всі	Checkout — оформлення замовлення
/profile	Всі	Profile — кабінет/авторизація
/sales	Всі	SalesPage — акції
/compare	Всі	ComparePage — порівняння
/support	Всі/Персонал	SupportPage — список тікетів
/support/:id	Auth	SupportTicket — чат тікету
/orders/:id	Auth	OrderPage — деталі замовлення
/admin	Manager+	AdminPanel — управління товарами
/admin/orders	Manager+	AdminOrders — замовлення
/admin/promo	Admin	AdminPromo — промокоди
/admin/users	Admin	AdminUsers — користувачі
/admin/support	Manager+	SupportPage (isAdmin) — підтримка